```
        case 'qux':
            print 'qux ';
            break;
    }
    ...
logicDemo('bar');
```

```
Output:
bar
```

Finally, just like a chain of `if`/`elseif` statements can have a final `else` block to execute if nothing else matches, `switch` has a `default` case. If we add one to the `switch` structure in `logicDemo()`…

```
        default:
            print 'default ';
            break;
    }
    ...
logicDemo('wibble');
```

```
Output:
default
```

… then we have a final alternative when all other cases fail to match.

Similar to `switch` is `match`, which allows a concise expression of multiple logical branches. There are, however, some important differences: `match` comparisons are strictly typed, they do not fall through, and they operate like functions in that they return values. However, don't be fooled into thinking `match()` is a function: it does not allow data typing of its single parameter, nor its "return" value. It also does not have a separate lexical scope; variables declared outside the `match()` block are accessible inside (indeed, they have to be). The following demonstrates the type strictness of `match()`:

```
$var = 1;
print match ($var) {
    '1' => 'string',
    1 => 'integer',
};
```